

- Devoir surveillé de mi semestre - Jeudi 13 novembre 2025

Groupe n° : Nom et prénom chinois : Prénom français : Note /100 :

La notation tiendra compte du soin de la copie et de la rédaction.

Exercice n° 1 : /40 points

Partie I : Algorithmique

- On cherche à écrire une fonction en Python, appelée `parcoursBFS`, de paramètres `M` de type array, et `s` de type int, qui parcourt en largeur le graphe G de matrice d'adjacences M à partir du sommet source s , les sommets de G étant numérotés $0, 1, \dots, n-1$, où n est la taille de la matrice M .

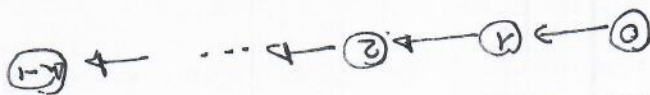
Compléter sur la feuille les « *** » de la fonction Python ci-dessous afin qu'elle renvoie la liste des sommets accessibles depuis le sommet source s à l'aide d'un parcours en largeur du graphe G .

```
def parcoursBFS(M,s):
    n=len(M)
    Access= ***           # *** remplacé par : [s]
    Avisiter= ***         # *** remplacé par : [s]
    while *** :          # *** remplacé par : Avisiter != []
        u=Avisiter.pop()
        L=[]
        for i in range(n):
            if *** :      # *** remplacé par : M[u,i] == 1
                L.append(i)
        # L est la liste de tous les voisins
        # (ou successeurs) de u
        for k in L:
            if *** :      # *** remplacé par : not (k in Access)
                ***       # *** remplacé par : Avisiter.insert(0,k)
            Access.append(k)
    return(Access)
```

2. Dans cette question, on ne demande pas de justifications.

(a) Quel est le nombre minimal de boucles `while` effectuées lorsque l'on exécute la fonction `parcoursBFS` ?

Pour $n \in \mathbb{N}^*$, donner un exemple de graphe orienté d'ordre n et d'un sommet source s associé pour lequel le nombre de boucles `while` effectuées est minimal.



Réponse : 1

Nombre minimal : 1

Représentation graphique d'un graphe orienté d'ordre n , avec pour sommet $s = n-1$:

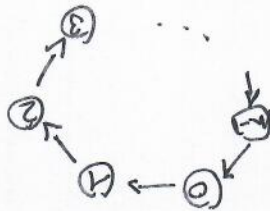
(b) Quel est le nombre maximal de boucles `while` effectuées lorsque l'on exécute la fonction `parcoursBFS` ?

Pour $n \in \mathbb{N}^*$, donner un exemple de graphe orienté d'ordre n et d'un sommet source s associé pour lequel le nombre de boucles `while` effectuées est maximal.

Réponse :

Nombre maximal : n

Représentation graphique d'un graphe orienté d'ordre n , avec pour sommet $s = 0$:



3. Les graphes en Python peuvent être définis sous la forme d'un dictionnaire, variable de type `dict`. Compléter alors les « *** » de la fonction Python ci-dessous, de paramètre `D` de type `dict`, afin qu'elle revête la matrice d'adjacences associée au graphe défini par le dictionnaire `D`.

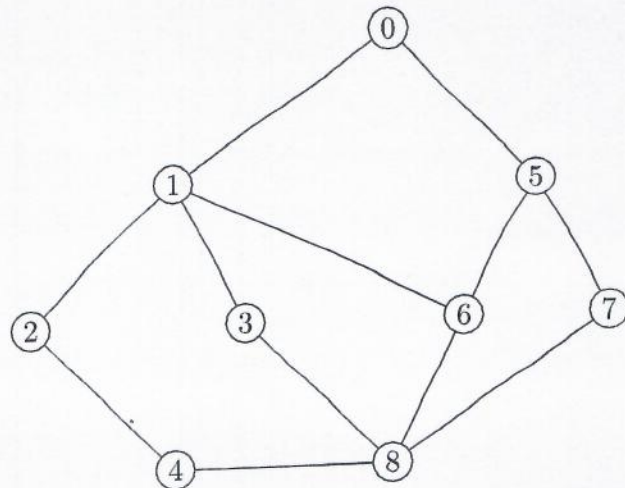
```
def traduction(D):
    n=len(D)
    L=D.keys()
    L=list(L)
    M=***
    for i in range(n):
        for j in range(n):
            u=L[i]
            v=L[j]
            if ****:
                M[i,j]=1
    return(M)
```

*** remplacé par : $v \in D[u]$

*** remplacé par : `np.zeros((n,n))`

Partie II : Un exemple

On considère le graphe G donné ci-contre :



4. Rappeler la définition du diamètre d'un graphe, puis préciser (sans justification) le diamètre de G .

Réponse :

$$\text{Si } G = (S, A), \text{ alors } \text{Diam}(G) = \max_{(u,v) \in S \times S} d(u,v).$$

$$\text{Ici } \text{Diam}(G) = 3$$

5. Compléter le tableau ci-dessous qui donne l'évolution des listes **Access** et **Avisiter**, définies dans la fonction Python **parcoursBFS** de la partie I, lorsque l'on applique l'algorithme de parcours en largeur au graphe G en partant du sommet source $s = 0$. Certains lignes pourront rester vides.

Étape initiale	Access= [0]	Avisiter= [0]
Étape n° 1	Access= [0, 1, 5]	Avisiter= [5, 1]
Étape n° 2	Access= [0, 1, 5, 2, 3, 6]	Avisiter= [6, 3, 2, 5]
Étape n° 3	Access= [0, 1, 5, 2, 3, 6, 7]	Avisiter= [7, 6, 3, 2]
Étape n° 4	Access= [0, 1, 5, 2, 3, 6, 7, 4]	Avisiter= [4, 7, 6, 3]
Étape n° 5	Access= [0, 1, 5, 2, 3, 6, 7, 4, 8]	Avisiter= [8, 4, 7, 6]
Étape n° 6	Access= [0, 1, 5, 2, 3, 6, 7, 4, 8]	Avisiter= [8, 4, 7]
Étape n° 7	Access= [0, 1, 5, 2, 3, 6, 7, 4, 8]	Avisiter= [8, 1]
Étape n° 8	Access= [0, 1, 5, 2, 3, 6, 7, 4, 8]	Avisiter= [8]
Étape n° 9	Access= [0, 1, 5, 2, 3, 6, 7, 4, 8]	Avisiter= []
Étape n° 10	Access=	Avisiter=

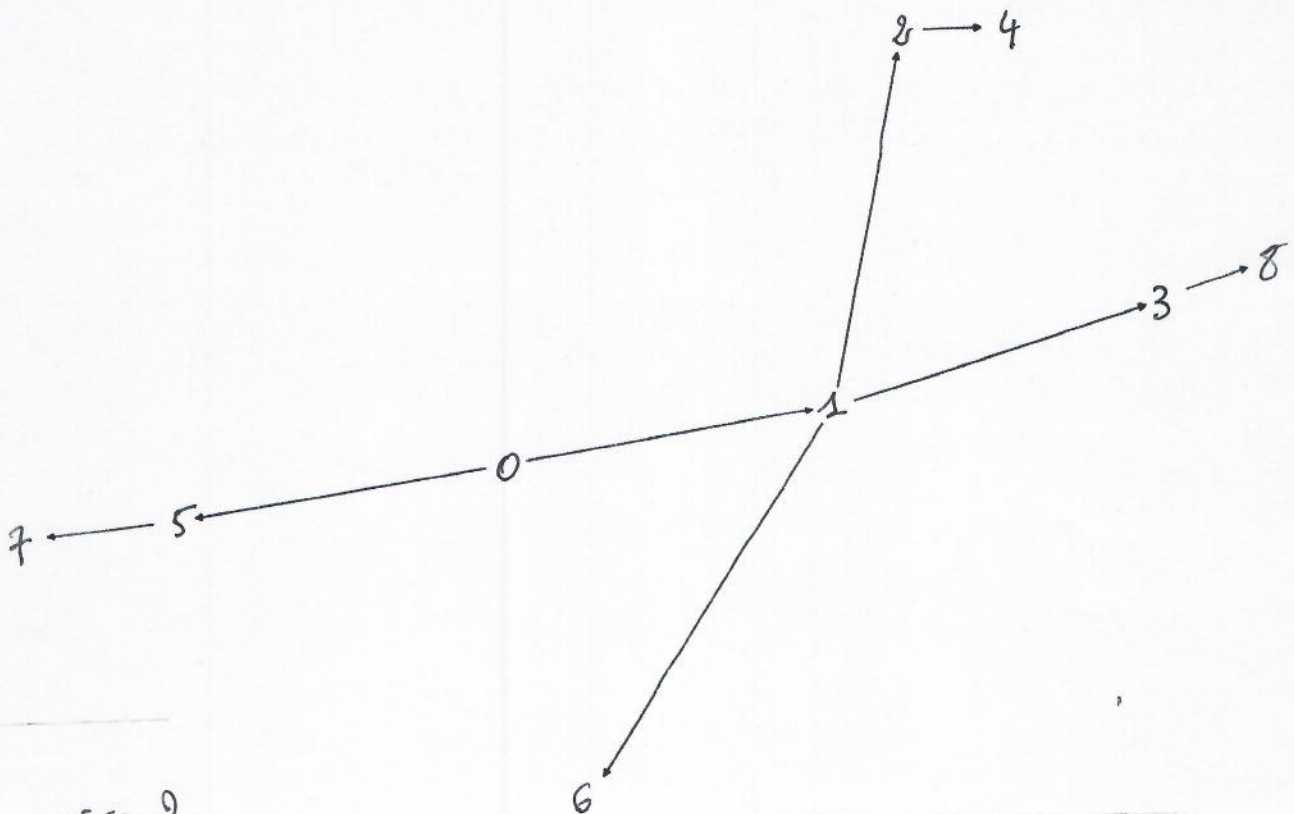
6. (a) Compléter les « *** » du script en Python ci-après, utilisant la fonction **traduction** de la partie I, afin de définir le graphe G et d'appliquer l'algorithme BFS fourni par la bibliothèque **networkx** de Python.

```

import numpy as np
import networkx as nx
D= ***      # *** remplacé par :  $D = \{0:[1,5], 1:[0,2,3,6],$ 
M=           $2:[1,4], 3:[1,6,8],$ 
            $4:[2,8], 5:[0,6,7],$ 
            $6:[1,5,8], 7:[5,8], 8:[3,4,6,7]\}$ 
G=nx.Graph(M)
Gb=nx.bfs_tree(G, source=0)
nx.draw(Gb)

```

- (b) Après exécution, on obtient la fenêtre graphique ci-dessous.
Compléter le graphique en écrivant les numéros dans les sommets correspondants.



Exercice 2

Partie I

1- G est connexe et G ne contient pas de cycle

Donc G est un arbre.

2) $\Delta = 3$ où $3 = \deg(3)$

3) $M = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{pmatrix}$

4) a) D'après le cours,

$$\chi_{\Pi}(\lambda) = \lambda^5 + c_1 \lambda^4 + c_2 \lambda^3 + c_3 \lambda^2 + c_4 \lambda + c_5$$

avec $c_1 = 0$

$c_2 = -m$ où m est le nombre d'arêtes de G

$c_3 = -2t$ où t est le nombre de triangles de G

Donc χ_{Π} est de la forme:

$$\chi_{\Pi}(\lambda) = \lambda^5 - 4\lambda^3 + a\lambda + b.$$

4b) $b = \det(\Pi)$

Π a deux colonnes identiques. Donc Π n'est pas inversible. Donc $\underline{b=0}$

4c) On a donc: $\chi_{\Pi}(\lambda) = \lambda^5 - 4\lambda^3 + a\lambda$

Avec les notations de 4a et d'après le cours:

$(-1)^4 c_4$ est la somme des mineurs diagonaux de M d'ordre 4.

Miner diagonal de M obtenu en enlevant

* la ligne et colonne n° 1: 0 (deux colonnes identiques)

* _____ 2: _____

* _____ 3: _____

* _____ 4: $\det \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} = (-1) \times \det \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} = 1$

* _____ 5: $\det \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} = 1.$

Donc $a = c_4 = 2$

5. Finalement $\chi_M(\lambda) = \lambda^5 - 4\lambda^3 + 2\lambda$
 $= (\lambda^4 - 4\lambda^2 + 2) \cdot \lambda$

$P(X) = X^2 - 4X + 2$ a pour discriminant $\Delta = 16 - 8 = 8$

P a pour racines : $\frac{4 \pm 2\sqrt{2}}{2} = 2 \pm \sqrt{2}$ et $2 + \sqrt{2}$

$1 < \sqrt{2} < 2$ Donc $2 - \sqrt{2} > 0$

Donc $Sp(M) = \{0, \sqrt{2-\sqrt{2}}, -\sqrt{2-\sqrt{2}}, \sqrt{2+\sqrt{2}}, -\sqrt{2+\sqrt{2}}\}$

6. * $\Delta = 3$.

* $\sqrt{2+\sqrt{2}} \leq 2\sqrt{2} \Leftrightarrow 2+\sqrt{2} \leq 8$
 $\Leftrightarrow \sqrt{2} \leq 6 : \text{vrai}$

* $\sqrt{2-\sqrt{2}} \leq \sqrt{2+\sqrt{2}} \leq 2\sqrt{2}$

Donc $Sp(M) \subset [-2\sqrt{\Delta-1}, 2\sqrt{\Delta-1}]$.

Partie II

7. Soient u et v deux sommets distincts de G .

Comme G est connexe, il existe une chaîne reliant u à v .

Quitte à parcourir la chaîne reliant u à v , il existe une chaîne élémentaire reliant u à v .

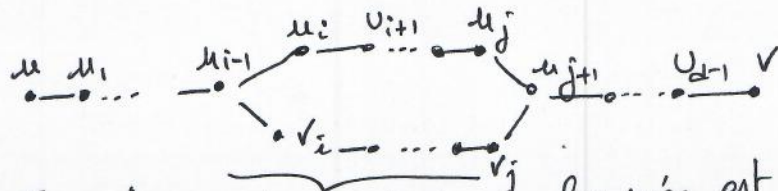
Supposons qu'il existe deux chaînes élémentaires distinctes,

notées $(u, u_1, \dots, u_{d-1}, v)$ et $(u, v_1, \dots, v_{e-1}, v)$, reliant u à v , avec $v = u_d$ et $v = v_e$.

Il existe au moins un sommet u_i et un sommet v_j tel que $u_i \neq v_j$.

Notons $i = \min \{ k \in \{1, \dots, d-1\} \mid u_k \neq v_k \}$
 et $j = \max \{ k \in \{i, i+1, \dots, d\} \mid u_k \neq v_k \}$

Alors :



Si l'un des u_k de la chaîne fermée est égal à l'un des v_k , on peut raccourcir la chaîne et obtenir un cycle, ce qui est absurde car G est un arbre.

Conclusion : Il existe une unique chaîne élémentaire reliant u à v .

8) Soit $(u, v) \in S^2$ tels que $d(u, v) = \text{Diam}(G)$

D'après 7, il existe une unique chaîne élémentaire, notée $(u, u_1, \dots, u_{d-1}, v)$, reliant u à v .

Par définition de la distance, on a donc : $d(u, v) = d$ (une plus courte chaîne reliant u à v étant nécessairement élémentaire)

Supposons que $\deg(v) > 1$

Notons u_0 un voisin de u tel que $u_0 \neq u_1$

Supposons la chaîne $(u_0, u, u_1, \dots, u_{d-1}, v)$ non élémentaire

Alors $\exists i \in \{2, \dots, d\} \mid u_0 = u_i$ (en notant $u_d = v$)

Alors $(u, u_0, u_{i+1}, \dots, u_d)$ est une chaîne élémentaire

reliant u à v . On obtient une contradiction d'après la

question 7.

Donc : $(u_0, u, u_1, \dots, u_{d-1}, v)$ est l'unique chaîne élémentaire reliant u_0 à v . Donc $d(u_0, v) = d+1 > \text{Diam}(G)$.

Il y a contradiction. Donc $\deg(v) \leq 1$. Et $\deg(v) > 0$ car G est connexe.

De même, $\deg(u) = 1$

9. Comme expliqué à la question 8, une chaîne réalisant la distance de r à s est nécessairement élémentaire.

On déduit de la question 7 que $l_u = d(r, u)$ si $u \neq r$.

Pour $u = r$, $l_r = 0$ et $d(r, r) = 0$.

10. Supposons que $\deg(s) \geq 2$.

Notons $(r, u_1, u_2, \dots, u_{l_s}, s)$ l'unique chaîne élémentaire reliant r à s .

Soit t un voisin de s .

Cas $t = u_{l_s-1}$. Alors $(r, u_1, \dots, u_{l_s-1})$ est une chaîne élémentaire reliant r à t .

Par unicité: $l_t = l_s - 1$.

Cas $t \neq u_{l_s-1}$ Comme à la question 8, $(r, u_1, \dots, u_{l_s-1}, s)$ étant la chaîne élémentaire la plus courte

reliant r à s , $\forall i \in \{1, \dots, l_s-1\}$, $t \neq u_i$ et $t \neq r$
 [Si $l_s = 1$, $r = u_{l_s-1}$ et $t \neq u_{l_s-1}$]

Donc $(r, u_1, \dots, u_{l_s-1}, s, t)$ est une chaîne élémentaire reliant r à t .

Par unicité: $l_t = d(r, s) + 1 = l_s + 1$.

11. La k^e ligne de $\pi X = \lambda X$ s'écrit

$$\sum_{j=1}^n m_{k,j} x_j = \lambda x_k$$

Donc: $|\lambda| \cdot \|X\|_\infty = \left| \sum_{j=1}^n m_{k,j} x_j \right| \leq \sum_{j=1}^n |m_{k,j}| \cdot |x_j|$

d'après l'inégalité triangulaire.

Donc $|\lambda| \|X\|_\infty \leq \sum_{j=1}^n |m_{k,j}| \|X\|_\infty$ car $\forall j \in \{1, \dots, n\}$,

Et $\|X\|_\infty \geq 0$ et $\sum_{j=1}^n |m_{k,j}| \leq \max_{1 \leq i \leq n} \left(\sum_{j=1}^n |m_{i,j}| \right)$ et $|m_{k,j}| \leq \|X\|_\infty$ et $|m_{k,j}| \geq 0$

Donc $|\lambda| \|X\|_\infty \leq \left(\max_{1 \leq i \leq n} \sum_{j=1}^n |m_{i,j}| \right) \times \|X\|_\infty$.

Comme $X \neq 0$ en tant que vecteur propre, $\|X\|_\infty > 0$

D'où $|\lambda| \leq \max_{1 \leq i \leq n} \sum_{j=1}^n |m_{i,j}|$.

12. * Comme $\Delta \geq 2$, $\forall i \in \{1, \dots, n\}$, $d_i > 0$

Donc la matrice diagonale D est bien inversible.

* Les matrices M et B sont semblables.

Donc $Sp(M) = Sp(B)$

* π est symétrique réelle car G est un graphe non orienté.

Donc π est diagonalisable, et en particulier: $Sp(n) \subset \mathbb{R}$

13. On a: $2 \times (\sqrt{\Delta-1})^2 = 2(\Delta-1)$

et $2(\Delta-1) \geq \Delta$, car $\Delta \geq 2$.

Donc $\Delta \leq 2\sqrt{\Delta-1} \times \sqrt{\Delta-1}$ et $\sqrt{\Delta-1} > 0$

Donc $\frac{\Delta}{\sqrt{\Delta-1}} \leq 2\sqrt{\Delta-1}$.

14) On note $\Pi = (m_{i,j})_{1 \leq i,j \leq n}$ et $B = (b_{i,j})_{1 \leq i,j \leq n}$.

On a: $\forall (i,j) \in \{1, \dots, n\}^2$,

$$b_{i,j} = \sum_{k=1}^n [D]_{i,k} \times \sum_{\ell=1}^n m_{k,\ell} [D^{-1}]_{\ell,j}$$

$$= d_i \times m_{i,j} \times \frac{1}{d_j} = \frac{(\sqrt{\Delta-1})^{l_{s_i} - l_{s_j}} m_{i,j}}{d_j}$$

15) Soit $i \in \{1, \dots, n\}$

D'après 14: $\sum_{j=1}^n |b_{i,j}| = \sum_{j=1}^n (\sqrt{\Delta-1})^{l_{s_i} - l_{s_j}} m_{i,j}$ car on rappelle que $m_{i,j} \in \{0, 1\}$

$$= \sum_{\substack{j=1 \\ s_j \text{ voisin de } s_i}}^n (\sqrt{\Delta-1})^{l_{s_i} - l_{s_j}}$$

Cas $\deg(s_i) = 1$ et $s_i \neq r$

Comme à la question 10, on montre que l'unique voisin s_j de s_i vérifie $l_{s_j} = l_{s_i} - 1$

Alors: $\sum_{j=1}^n |b_{i,j}| = \sqrt{\Delta-1} \leq \underline{2\sqrt{\Delta-1}}$ car $\sqrt{\Delta-1} \geq 0$

Cas $\deg(s_i) = 2$ et $s_i \neq r$

Notons $\{t_1, \dots, t_d\}$ les voisins de s_i , avec $d = \deg(s_i)$, $l_{t_1} = l_{s_i} - 1$ et $\forall j \in \{2, \dots, d\}, l_{t_j} = l_{s_i} + 1$

Alors:

$$\sum_{j=1}^n |b_{ij}| = \sqrt{\Delta-1} + (d-1) \times (\sqrt{\Delta-1})^{-1}$$

$$\leq \sqrt{\Delta-1} + (\Delta-1) \frac{1}{\sqrt{\Delta-1}} \quad \begin{array}{l} \text{car } d = \deg(s_i) \\ \leq \Delta, \\ \text{par définition de } \Delta. \end{array}$$

$$= 2\sqrt{\Delta-1}$$

Cas $s_i = r$

Comme $\deg(s_i) = 1$, et que $l_t = 1$ pour t voisin de r ,
alors $\sum_{j=1}^n |b_{ij}| = \frac{1}{\sqrt{\Delta-1}} \leq \frac{\Delta}{\sqrt{\Delta-1}} \leq 2\sqrt{\Delta-1}$

L'inégalité est prouvée dans tous les cas.

16. Soit $\lambda \in \text{Sp}(M)$. Alors $\lambda \in \text{Sp}(B)$ d'après 12.

D'après 11: $|\lambda| \leq \max_{1 \leq i \leq n} \sum_{j=1}^n |b_{i,j}|$

D'après 15: $\max_{1 \leq i \leq n} \sum_{j=1}^n |b_{i,j}| \leq 2\sqrt{\Delta-1}$

Donc $|\lambda| \leq 2\sqrt{\Delta-1}$

Donc $\text{Sp}(M) \subset [-2\sqrt{\Delta-1}, 2\sqrt{\Delta-1}]$.